

Introdução — Apresentação da disciplina

Resumo 1 de 14 · arquivo 0apresentação · 31 slides

Este é o slide de abertura da disciplina. Ele tem duas metades: primeiro as **regras e informações do curso** e depois uma longa **motivação** que responde à pergunta “por que precisamos de sistemas distribuídos?”. Para quem nunca viu o termo: um **sistema distribuído** é um conjunto de vários computadores independentes que cooperam pela rede e, para o usuário, parecem um sistema único. Toda a segunda metade da aula existe para justificar por que isso se tornou necessário.

1. Parte administrativa

- **Contato:** Prof. Daniel Cordeiro (daniel.cordeiro@usp.br, sala I1-352D); materiais na plataforma **e-Disciplinas** da USP.
- **Aviso sobre multitarefa:** citando pesquisa de Stanford (Clifford Nass), argumenta-se que fazer várias coisas ao mesmo tempo prejudica atenção e memória — as distrações digitais liberam pequenas doses de dopamina que viciam, e quem faz multitarefa gasta mais esforço cerebral que quem foca numa coisa só. **Conclusão prática:** nada de celular/notebook em aula.
- **Curso presencial:** pelo Regimento da USP, aprova quem tiver nota final ≥ 5 e frequência $\geq 70\%$.
- **Acessibilidade:** alunos com deficiência (visual, física, auditiva, dislexia etc.) devem avisar o professor o quanto antes.
- **Livro-texto:** *Distributed Systems*, 4ª edição (2023), de Maarten van Steen e Andrew Tanenbaum — a referência clássica da área. Os **tópicos do curso** (que definem toda a trilha de aulas) são: Introdução, Arquiteturas, Processos, Comunicação, Nomes, Coordenação, Replicação, Tolerância a falhas e Segurança.

2. Por que precisamos de sistemas distribuídos?

São apontadas **quatro classes de problemas** que ainda exigem inovação constante: (1) problemas “em escala da web”, (2) grandes data centers, (3) computação paralela e distribuída, e (4) aplicações web interativas.

Problemas em escala da web

São, antes de tudo, **data-intensive** (envolvem volumes gigantescos de dados) e às vezes também **processing-intensive** (exigem muito cálculo). Exemplos: rastrear e indexar a web (*crawling*), buscas, mineração de dados, bioinformática pós-genômica, dados científicos (física, astronomia), redes de sensores e aplicações Web 2.0.

O volume de dados

Dá a dimensão do problema: estima-se que o mundo tinha 44 zettabytes de dados no fim de 2020 e produziria 463 exabytes *por dia* até 2025; gigantes como Google, Facebook, Microsoft e Amazon guardam ao menos 1.200 petabytes cada. As unidades crescem em fatores de mil: giga 10^9 , tera 10^{12} , peta 10^{15} , exa 10^{18} , zetta 10^{21} .

O que se faz com tantos dados?

Extraír fatos e descobrir relações novas via **casamento de padrões**. O exemplo dado: a partir de fatos como “Mozart nasceu em 1756” e “Einstein nasceu em 1879”, o sistema deduz o padrão genérico “PESSOA nasceu em DATA”, que por sua vez permite descobrir automaticamente novos fatos no texto da web. A estratégia geral é **dividir para conquistar** e ir **adicionando mais recursos** conforme os dados crescem.

3. Grandes data centers

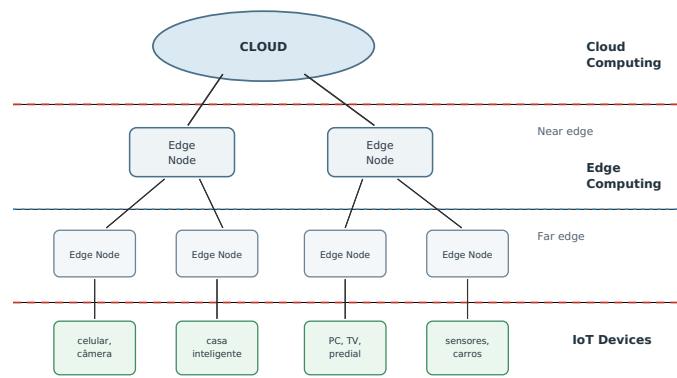
Os sistemas que afetam quase todo mundo na Internet rodam em data centers enormes. Só o **Google tinha 38 data centers** espalhados pelo mundo (fev/2025). Isso tem um custo energético crescente: segundo a Agência Internacional de Energia (IEA), os data centers consumiram cerca de **460 TWh em 2022 — quase 2% de toda a eletricidade do planeta** — e a inteligência artificial e as criptomoedas podem **dobrar** essa demanda até 2026 (acréscimo equivalente ao consumo de um país inteiro).

4. Como chegamos até aqui (evolução da computação)

Linha do tempo: **anos 50** (computadores eram grandes calculadoras com cartões perfurados; início da computação paralela) → **fim dos anos 60** (ARPANET interliga computadores; nasce a ideia de “computação como serviço”) → **anos 70** (microprocessadores) → **anos 80** (computadores pessoais) → **anos 90** (popularização da Internet).

Os **paradigmas de computação** que surgiram: computadores pessoais, computadores paralelos, aglomerados/*clusters*, **computação voluntária** (pessoas doam o tempo ocioso de seus PCs — ex.: GIMPS/1996 procurando primos de Mersenne, distributed.net/1997 quebrando criptografia por força bruta, SETI@Home/1999 buscando sinais de vida extraterrestre) e **computação em grade** (*grid*).

Hoje isso desemboca no **cloud continuum**, uma hierarquia em camadas do centro (nuvem) até a borda (dispositivos). A ideia é que o processamento não fica só no centro: parte é empurrada para nós intermediários (“edge”), mais perto dos dispositivos, reduzindo a latência.



Cloud continuum: hierarquia da nuvem (centro) até os dispositivos IoT (borda), passando por nós “edge”.

5. Problemas atuais e ideias-chave

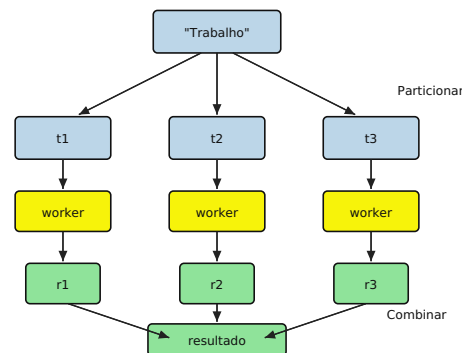
Quando o problema cresce na escala da web, a resposta “fácil” é **adicionar mais máquinas**, o que leva à centralização em grandes data centers. A localização deles não é acidental: fiordes noruegueses, Islândia, Oregon e minas abandonadas têm em comum **clima frio e/ou energia barata** — ótimos para refrigerar milhares de servidores. Os desafios então passam a ser **redundância, eficiência, utilização e gerenciamento**.

Três ideias norteiam o projeto desses sistemas:

- **Escalabilidade horizontal, não vertical** — em vez de comprar uma máquina cada vez maior (vertical, que tem limite físico), acrescentam-se muitas máquinas comuns (horizontal).
- **Mover o processamento para perto dos dados** — porque a banda de rede é limitada; é mais barato levar o código até o dado do que o contrário.
- **Processar dados sequencialmente** — evitar acessos aleatórios ao disco, pois os saltos de posição (*seeks*) são caros, enquanto a vazão sequencial é boa.

6. Como programar aplicações escaláveis

O padrão central é **divisão e conquista**: quebra-se o trabalho em pedaços, cada pedaço é processado por um *worker* em paralelo, e os resultados parciais são recombinaos.



Divisão e conquista: o trabalho é particionado em tarefas (t1...t3), processado por workers em paralelo e os resultados parciais (r1...r3) são combinados.

Isso levanta vários **desafios de paralelização**: como repartir as unidades de trabalho? o que fazer quando há mais trabalho do que workers? e se os workers precisarem compartilhar resultados intermediários? como agregar os resultados parciais? o que fazer se um worker falhar? como saber que todos terminaram?

O **problema recorrente** é que esses percalços surgem da **comunicação entre workers** e do **acesso a recursos compartilhados** (por exemplo, os dados). Por isso é preciso algum **mecanismo de sincronização**.

Gerenciar múltiplos workers é difícil porque não sabemos a ordem de execução, quando um interromperá o outro, nem em que ordem acessarão os dados compartilhados. As ferramentas de sincronização clássicas são **semáforos** (lock/unlock), **variáveis condicionais** (wait/notify/broadcast) e **barreiras**. Mesmo assim restam problemas como *deadlock* (impasse), *starvation* (inanição) e *race conditions* (condições de corrida).

As **ferramentas atuais** incluem dois modelos de programação — **memória compartilhada** (ex.: pthreads) e **passagem de mensagens** (ex.: MPI) — e padrões arquiteturais como mestre-escravo, produtor-consumidor e filas de trabalho compartilhadas.

Moral da história: tudo se resume a escolher o **nível de abstração adequado** — esconder os detalhes do sistema do desenvolvedor evita boa parte desses problemas. Deve-se **separar o “quê” do “como”**: o desenvolvedor diz apenas o que computar, e o arcabouço (framework) cuida de como executar. O lema resume: **“O data center é o computador!”**

7. Exemplo prático (causo): Animoto

A Animoto, uma startup que gera vídeos a partir de fotos, clipes e música, disponibilizou seu serviço no Facebook e sofreu uma **explosão de demanda**: precisou saltar de **50 para 3.500 servidores em três dias**; depois desse pico, o tráfego caiu para um nível muito menor. Uma empresa tradicional (com servidores próprios) teria ficado sobrecarregada ou desperdiçado dinheiro. Com **computação em nuvem**, paga-

se mais durante os picos e devolvem-se os recursos ociosos depois. O gráfico da aula confirma a história: o número de instâncias EC2 (servidores alugados na Amazon) fica estável perto de ~50 por vários dias e então dispara para cerca de 3.400-3.500 em pouco tempo — exatamente o comportamento elástico que a nuvem permite.